

SOFTWARE/WEB SERVER ARTICLE

BioPipelines: Accessible Computational Protein and Ligand Design for Chemical Biologists

Gianluca Quargnali* and Pablo Rivera-Fuentes*

Department of Chemistry, University of Zurich, 8057 Zurich, Switzerland.

*Address correspondence to: gianluca.quargnali@chem.uzh.ch (G.Q.); pablo.riverafuentes@chem.uzh.ch (P.R.-F.)

Deep learning methods for protein structure generation, sequence design, and structure and property prediction have created unprecedented opportunities for protein engineering and drug discovery. However, using these tools often requires navigating incompatible software environments, diverse input/output formats, and high-performance computing infrastructure, any of which may hinder adoption by primarily experimental chemical biology laboratories. Here, we present BioPipelines, an open-source Python framework that allows researchers to define multistep computational design workflows in a few lines of code. Additionally, its robust yet modular architecture provides a straightforward way to expand the tool kit with different functionalities, particularly by leveraging coding agents, with little effort. The framework currently integrates over 40 tools encompassing structure generation, sequence design, structure prediction, compound screening, and analysis. The same workflow code can be prototyped interactively in a Jupyter notebook and then submitted for production-scale runs without modification. We demonstrate applications in inverse folding, gene synthesis, *de novo* protein design, compound library screening, iterative binding site optimization, and fusion-protein linker optimization. We hope that this framework will empower researchers, allowing them to focus on the scientific question rather than computational logistics. BioPipelines is available under the MIT license at <https://github.com/locbp-uzh/biopipelines>.

Introduction

Over the past decade, and especially with the advent of deep learning models, protein engineering has shifted from complex, expert-oriented software suites like Rosetta [1] to a scattered collection of tools designed to work in a “black-box” fashion with simple inputs. AlphaFold2 [2], which takes only a protein sequence as input and yields a structural prediction in output, exemplifies this trend. Subsequent structure prediction methods, including AlphaFold3 [3], RoseTTAFold3 [4], Chai-1 [5], and Boltz-2 [6] operate similarly and have extended prediction to biomolecular complexes encompassing proteins, nucleic acids, and small molecules, with the latter also providing binding affinity estimation. Diffusion-based methods, including RFdiffusion [7], Chroma [8], and FrameDiff [9], have made backbone generation routine, producing scaffolds with conditioned structural features. In parallel, inverse folding models, notably ProteinMPNN [10], have enabled rapid sequence design for target structures, with extensions such as LigandMPNN [11] accommodating small-molecule binding partners. Dozens of other tools now exist, including tools for docking and protein–ligand interaction analysis, among others.

These advances are especially relevant to chemical biology, where researchers routinely need to engineer proteins with altered binding specificity, design enzyme variants for new substrates, or screen compound libraries against protein targets. However, a framework is lacking to systematically connect these tools in a simple, standardized way. A typical computational

campaign might involve generating protein scaffolds around a ligand binding site, designing sequences compatible with that scaffold, predicting the structures of the resulting designs, and ranking them by predicted binding affinity. In practice, however, chaining these tools together presents substantial challenges for laboratories without dedicated computational support. Each tool requires its own software environment, uses different input and output file formats, and must be configured for the computing cluster. Running a multi-tool workflow manually involves writing and debugging shell scripts, tracking intermediate files across tools, and managing job dependencies on the cluster scheduler. These logistical hurdles are often the rate-limiting step in adopting computational design.

Some workflow frameworks have been developed to address this problem. ColabFold pioneered the democratization of protein folding tools [12], but it has remained largely focused on structure prediction models. Ovo [13] offers a web interface built on the Nextflow [14] workflow engine but requires a database server and containerization infrastructure. ProteinDJ [15] achieves efficient multi-graphics processing unit (multi-GPU) parallelism but restricts users to 9 predefined pipeline configurations without support for custom workflows or iterative optimization. ProtFlow [16] provides Python wrappers around design tools with cluster job management, but the absence of typed data for common entities in chemical biology impairs its modularity. Moreover, it requires writing verbose configuration code and maintaining a running Python process throughout execution.

Citation: Quargnali G, Rivera-Fuentes P. BioPipelines: Accessible Computational Protein and Ligand Design for Chemical Biologists. *Comput. Struct. Biotechnol. J.* 2026;35(1):Article 0129. <https://doi.org/10.34133/csbj.0129>

Submitted 23 March 2026

Revised 14 May 2026

Accepted 14 May 2026

Published 8 June 2026

Copyright © 2026 Gianluca Quargnali and Pablo Rivera-Fuentes. No claim to original U.S. Government Works. Distributed under a Creative Commons Attribution License (CC BY 4.0).

Here, we present BioPipelines, a framework designed to make computational protein and ligand design accessible to research groups with minimal computational expertise. In BioPipelines, workflows are expressed in a compact, experiment-like syntax in which tools interact through standardized biomolecular data streams. Pipelines can be inspected and tested interactively, for example, in Jupyter, and be submitted to a SLURM [17] computing cluster without modification. Further, the framework can easily be extended to incorporate diverse tools, particularly by leveraging artificial intelligence (AI) coding agents. We demonstrate these capabilities through a series of application examples spanning common tasks in chemical biology.

Results and Discussion

Software design

BioPipelines was designed with 3 objectives in mind: (a) abstraction: pipelines read like a description of an experiment and run with minimal user-side tuning of the underlying computational infrastructure; (b) modularity: arbitrary tools can be implemented following simple, biomolecular-informed rules, so that they can serve as both input and output to pre-existing tools according to a standardized interface; and (c) testability: pipelines can be tested in interactive environments like Jupyter notebooks, and intermediate results can be easily inspected. We briefly discuss the main implementation here before focusing on some usage case examples and redirect the interested reader to the Supplementary Materials, where a technical description of the architecture is provided.

Abstraction is achieved by separating configuration and execution into 2 conceptually distinct and subsequent phases. The configuration phase is fully orchestrated in Python, the most common high-level programming language in biochemical computational tools. A workflow is written as a compact script defining which tools to run, with what parameters, and how their outputs connect. At this stage, the framework predicts the file system structure and outputs that will result after execution and generates self-contained bash scripts handling tools execution and interfacing. Combined with a straightforward and standardized syntax rooted in Python's context manager, it allows users to write pipelines in seemingly natural language, as demonstrated in the coming examples. Only in the second phase, after the Python script has terminated, the ensemble of bash scripts is executed. Therefore, no long-running orchestrator is required during cluster execution. The generated scripts can be inspected, modified, or resubmitted independently, serving as both execution artifacts and documentation of what was run.

The modularity of BioPipelines stems from a standardized representation of the entities on which tools operate and which tools produce. We identified a set of 3 basic types, which are (a) structures (2-dimensional/3-dimensional [3D] data: .pdb, .cif, .sdf, etc.), (b) sequences (1-dimensional data: proteins, DNA, and RNA), and (c) compounds (Simplified Molecular Input Line Entry System [SMILES], Chemical Component Dictionary, etc.). These constitute the primary item streams flowing between different tools in a pipeline, together with tabular data associated with them. Nearly all tools of interest for chemical biology applications operate on one or more of these and, provided some tool-specific parameters, produce another set. For example, ProteinMPNN takes in structures

and yields sequences, AlphaFold2 takes in sequences and yields structures, and Boltz-2 takes in sequences and compounds and yields structures. Any such tool can be integrated into BioPipelines by writing an associated tool class that predicts its output from its input. To complete the picture, BioPipelines allows arbitrary definitions of streams. For example, Boltz-2 generates multiple sequence alignment (MSA) files when they are not provided, and these can be recycled in subsequent runs. BioPipelines provides standardized functions to handle streams and operations on associated data tables like filtering and sorting.

Finally, testability is greatly enhanced by the fact that the same pipeline code can execute in on-the-fly mode when running in a Jupyter or Google Colab notebook. BioPipelines auto-detects the interactive environment and executes each tool immediately upon instantiation, streaming the console output to the notebook. Given that tool outputs are standardized, tools have a default implemented visual representation in notebooks: structures streams are displayed as interactive MOL files, plots are displayed in line, and so on. Compute-heavy steps can run on a cluster or Colab GPU, while the standardized output tables and streams can be downloaded and postprocessed locally. Filtering, sorting, and geometric analyses, as well as their corresponding visualizations, all run without the need for a high-performance GPU.

We have summarized the organization of the framework into 4 conceptual stages in Fig. 1. In the following sections, we showcase a few selected applications of BioPipelines to demonstrate its current capabilities, the simplicity and modularity of the code, and how these capabilities can be easily extended to include new tools. The examples are chosen to span common chemical biology tasks and to illustrate different patterns of information flow between tools. Importantly, we make no claims to the scientific accuracy of any of the implemented tools. We specifically alert the reader to the fact that cofolding tools used for affinity ranking have known reliability limits that must be considered before experimental follow-up [18–20].

Application 1: Redesigning the sequence of a protein

The sequence of proteins largely dictates their physical properties, such as stability and solubility. Hence, it is of interest to design mutants that preserve the fold of the original protein but may lead to improved physical properties. BioPipelines can be used to generate alternative sequences of a given protein. In the example pipeline (Fig. 2) [21], we generate alternative sequences for ubiquitin. After defining the computational resources needed for the job, the starting structure of the protein is loaded from the Protein Data Bank (PDB: 4LCD) [22]. The sequence is redesigned with the ProteinMPNN model trained on soluble proteins, and the resulting sequences are folded with AlphaFold2 for inspection. Finally, these sequences are passed through a DNA encoder to produce codon-optimized DNA sequences ready for synthesis, cloning, and expression testing in *Escherichia coli*. The DNAEncoder tool uses organism-specific codon usage tables from CoCoPUTs [23], applying, as a default option, a thresholded weighted sampling to avoid rare codons while reducing gene repeats.

Instead of relying on manual inspection of the structures, analysis and filtering within the pipeline are also supported. Commonly, one would discard folds that do not resemble the original or that AlphaFold2 predicts with low confidence. This can easily be done with integrated tools in BioPipelines, by appending them to the workflow. For example, we can compute

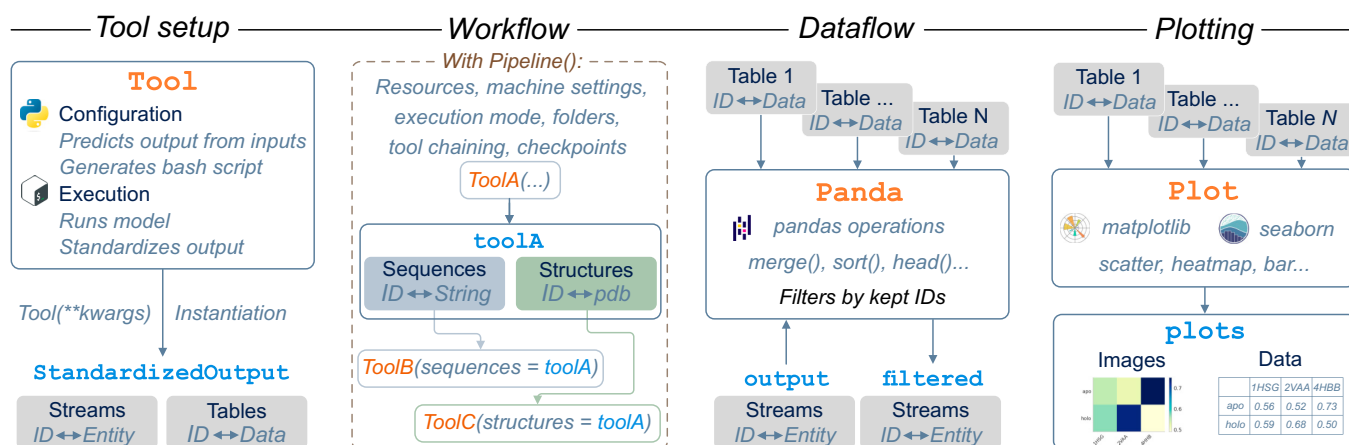


Fig. 1. Architecture and data handling. Tools wrap external programs and produce standardized streams of entities and tables of associated data. Pipelines compose tools so that the output of one becomes the input of the next. The Panda tool performs tabular operations and can filter standardized outputs, both streams and tables, by the IDs surviving the operations. The Plot tool generates images together with the underlying data as a csv table.

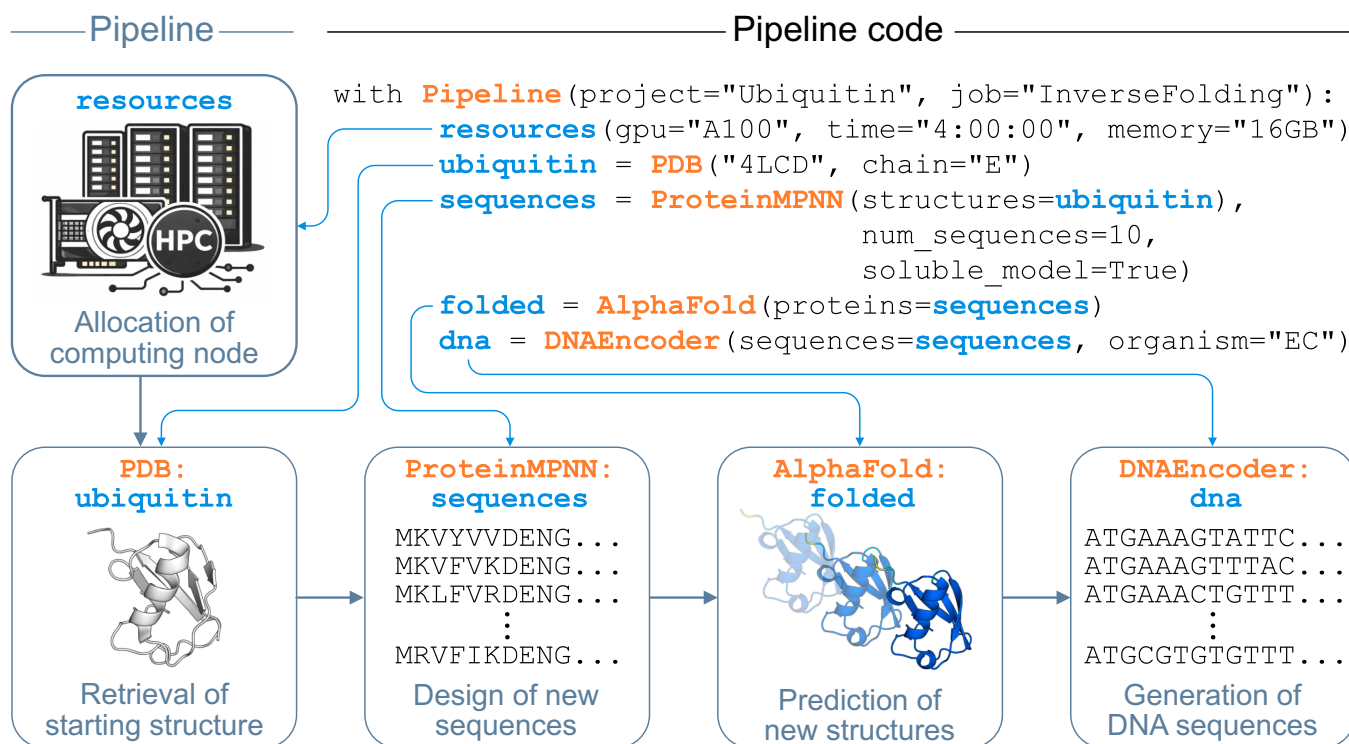


Fig. 2. Pipeline and code for the design of alternative sequences of ubiquitin. BioPipelines' Python classes are highlighted in orange and user-defined objects in blue.

the root mean square deviation of each new protein with respect to the parent protein and combine this information with a confidence metric from AlphaFold2, like the predicted local distance difference test, to obtain a refined set of sequences (Fig. S1). Table operations, as well as ranking, filtering, and sorting based on tabular data, are handled by the BioPipelines tool Panda, which relies on the widespread Python package pandas [24].

Application 2: Redesigning a protein domain

Generative tools allow for designing entirely new proteins or parts of existing proteins, with countless potential applications in protein engineering, antibody design, and synthetic biology. BioPipelines can be used to seamlessly integrate *de novo* design

of backbones, inverse folding, structure prediction, and analysis of results allowing the user to choose the specific model for each step of the design process. In the following example, the LID, a nonessential domain of the protein adenylate kinase (PDB: 4AKE), is redesigned. The pipeline is very similar to the previous example, starting with the allocation of computing resources and retrieval of the starting structure from the PDB (Fig. 3). The only main difference is that RFdiffusion is called to generate 10 new protein backbones, replacing the segment A118-160 (LID) with new backbones of lengths between 50 and 70 amino acids. After this step, the workflow continues as for the ubiquitin case, with inverse folding of the new backbones by ProteinMPNN, which generates 2 sequences for each backbone generated by RFdiffusion, and structure validation by AlphaFold2.

```
with Pipeline(project="Kinase", job="LID_Redesign"):
    resources(gpu="A100", time="4:00:00", memory="16GB")
    kinase = PDB("4AKE")
    backbones = RFdiffusion(pdb=kinase,
                           contigs='A1-117/50-70/A161-214',
                           num_designs=10)
    sequences = ProteinMPNN(structures=backbones,
                           num_sequences=2,
                           redesigned=backbones.tables.structures.designed)
    refolded = AlphaFold(proteins=sequences)
```

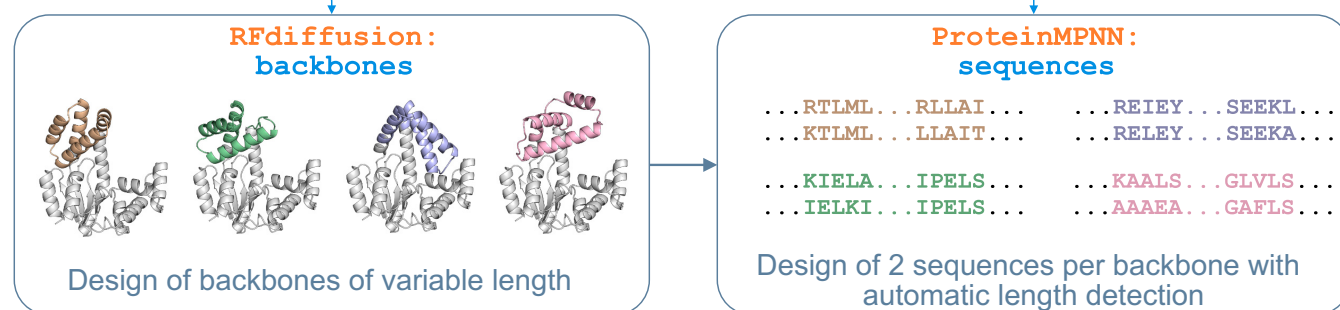


Fig. 3. BioPipelines' succinct implementation of the classic RFdiffusion–ProteinMPNN–AlphaFold2 [AF2] pipeline. The resource allocation, starting structure retrieval (Protein Data Bank [PDB] class), and refolding (AlphaFold class) steps are equivalent to those depicted in Fig. 1.

The `redesigned=backbones.tables.structures.designed` argument illustrates how information flows between tools. Each backbone has different designed positions, determined by contig sampling of RFdiffusion at runtime, and this per-structure information is passed to ProteinMPNN without any manual file parsing, thus ensuring that ProteinMPNN designs sequences only for the newly generated RFdiffusion backbone, regardless of its length.

Pipelines involving RFdiffusion usually require hundreds, if not thousands, of designs to produce high-quality hits, depending on the objective. We can use the Panda tool again to not only filter the designs but also sort them based on any given metric produced and select the best ones. In Fig. S2, we show how to proceed from the pipeline to filter based on the conformational change of the nondesigned portion of the protein, sort by model confidence, and generate a PyMOL session (.pse) file in which the top 3 structures are colored according to the AlphaFold coloring scheme for confidence in the designed regions and white otherwise.

Application 3: Screening a library of compounds against a target protein

Computational screening of compound libraries against protein targets is a common need in chemical biology, particularly for identifying lead compounds or optimizing binding interactions. BioPipelines provides tools for building combinatorial compound libraries and interface them with structure prediction tools.

Boltz-2 can be used for the cofolding of ligands with diverse biomolecules (proteins, DNA, and RNA) and at the same time provides a binding probability prediction (classification) and an associated binding affinity (regression). In the following

pipeline (Fig. 4), a small library of tryptophan derivatives is screened against the homodimer of tryptophan repressor (TrpR) in the presence of its DNA operator (PDB: 1TRO). It should be noted that recent benchmarks of cofolding methods have documented limitations that the user should keep in mind: predicted poses can violate basic physical-validity criteria, generalization to protein targets dissimilar from the training set is limited, allosteric ligands are frequently misplaced into the orthosteric pocket over-represented in the PDB, and the affinity regression is less reliable than the binary classifier for ranking docking hits [18,19,25,26]. BioPipelines exposes the PoseBusters [20] tool that could be appended after any cofolding step to flag predictions that fail standard chemical and physical-validity checks, so that downstream selection can be restricted to physically plausible poses.

In BioPipelines, Boltz-2 has been integrated into a simple syntax for declarative control over which entities and modifications to include while maintaining clarity through the combinatorial assistants Bundle and Each. In the above expression, we are instructing Boltz-2 to cofold 2 copies of TrpR with each ligand present in the library, resulting in 3 separate predictions. This flexible syntax is maintained for more nested scenarios, for example, to calculate the affinity of each ligand in the presence of the homodimer and another molecule, or vice versa. The compound library can be generated combinatorially either from a ChemDraw file with appropriate R-group tables or from a SMILES dictionary with declared branching points (Fig. S3).

Finally, information flow allows us to easily gather useful metrics. In Fig. S4, we use the Plot tool to plot the affinity predicted by Boltz-2 against the R1 substituent from the library: the table “compounds” contains all the branching information for each compound. Importantly, the Plot tool generates a csv table containing the underlying data associated with each image file, so users can easily replot using their favorite software.

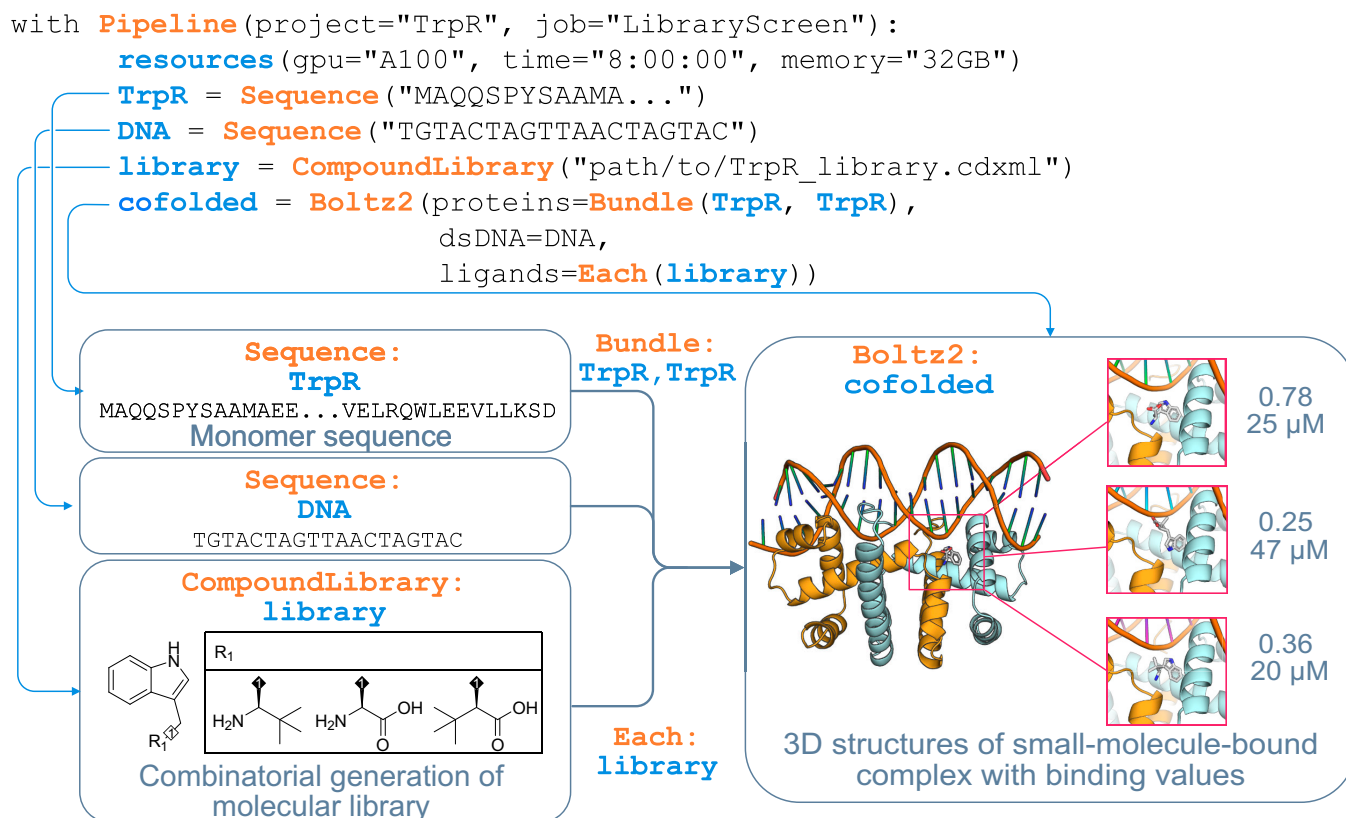


Fig. 4. Definition of a branched compound library and interface with Boltz-2. Bundle and Each control how to include entities into predictions: with Bundle on 2 copies of tryptophan repressor (TrpR), we have both inside each prediction, resulting in a homodimer; with Each on the compound library, we have one prediction for each of the compounds. Affinities are provided as probability (unitless) and binding affinity (μM).

Application 4: Modeling a Förster resonance energy transfer calcium sensor

Genetically encoded Förster resonance energy transfer (FRET) sensors typically sandwich a conformationally responsive sensing domain between donor and acceptor fluorescent proteins. A critical design question is which linkers keep both fluorescent domains properly folded while maximizing the change in FRET efficiency upon ligand binding. BioPipelines enables systematic screening of linker variants with structural prediction under both apo and holo conditions. For example, the following pipeline (Fig. 5) constructs a calmodulin-based calcium sensor by fusing the enhanced blue fluorescent protein (donor), calmodulin (sensing domain), and the enhanced yellow fluorescent protein (acceptor) with variable-length flexible linkers. Both apo and Ca^{2+} -bound forms are predicted with Boltz-2, and the chromophore distance and interdomain orientation are compared between states. A comprehensive screening, including both linker length and sequence, could easily be applied by using the Mutagenesis tool (Fig. S5), and more structured linkers could be designed using RFDiffusion and ProteinMPNN. In the simple case shown here, the Fuse tool generates all linker-length combinations ($4 \times 4 = 16$ constructs), concatenating the 3 domains with short flexible linkers. Both apo and holo forms are predicted for each construct, and MSAs required for folding are recycled between the 2, reducing impact on the server.

Further analysis of the designs could be carried out using the Distance tool (Fig. S6) to measure the chromophore separation (residue 66 in the donor, residue -173 counting from the

C-terminus in the acceptor), which is considered to be the most determinant geometric parameter for FRET efficiency when fluorophores are not rotationally restricted, an assumption that we are making in this example. BioPipelines also provides the Angle tool for calculating angles and dihedrals, which could be used to extend the pipeline to measure interdomain orientation. Finally, the Plot tool is used to visualize which linker combinations produce the largest predicted calcium-dependent change in FRET geometry, providing a starting point for experimental sensor optimization.

Beyond simple design pipelines: Iterative metric optimization

Many protein engineering campaigns can benefit from multiple rounds of design and evaluation, selecting a candidate for further engineering in the next iteration (e.g., directed evolution). With its modular nature, simple syntax, and tool-agnostic information flow, BioPipelines is a suitable platform to develop metric optimization methods. Our example pipeline demonstrates a simple case of cycles of optimization in a compact syntax; however, more sophisticated optimization schemes, such as machine-learning-guided directed evolution [27,28], Bayesian optimization [29,30], active learning [31–33], or reinforcement learning [34], could also be implemented.

In our example pipeline (Fig. 6), the objective is to find alternative sequences for the binding pocket of the periplasmic binding protein NocT that bind histopine noncovalently. The wild-type complex is first cofolded with Boltz-2 to provide a starting structure and a reference predicted affinity. Each subsequent

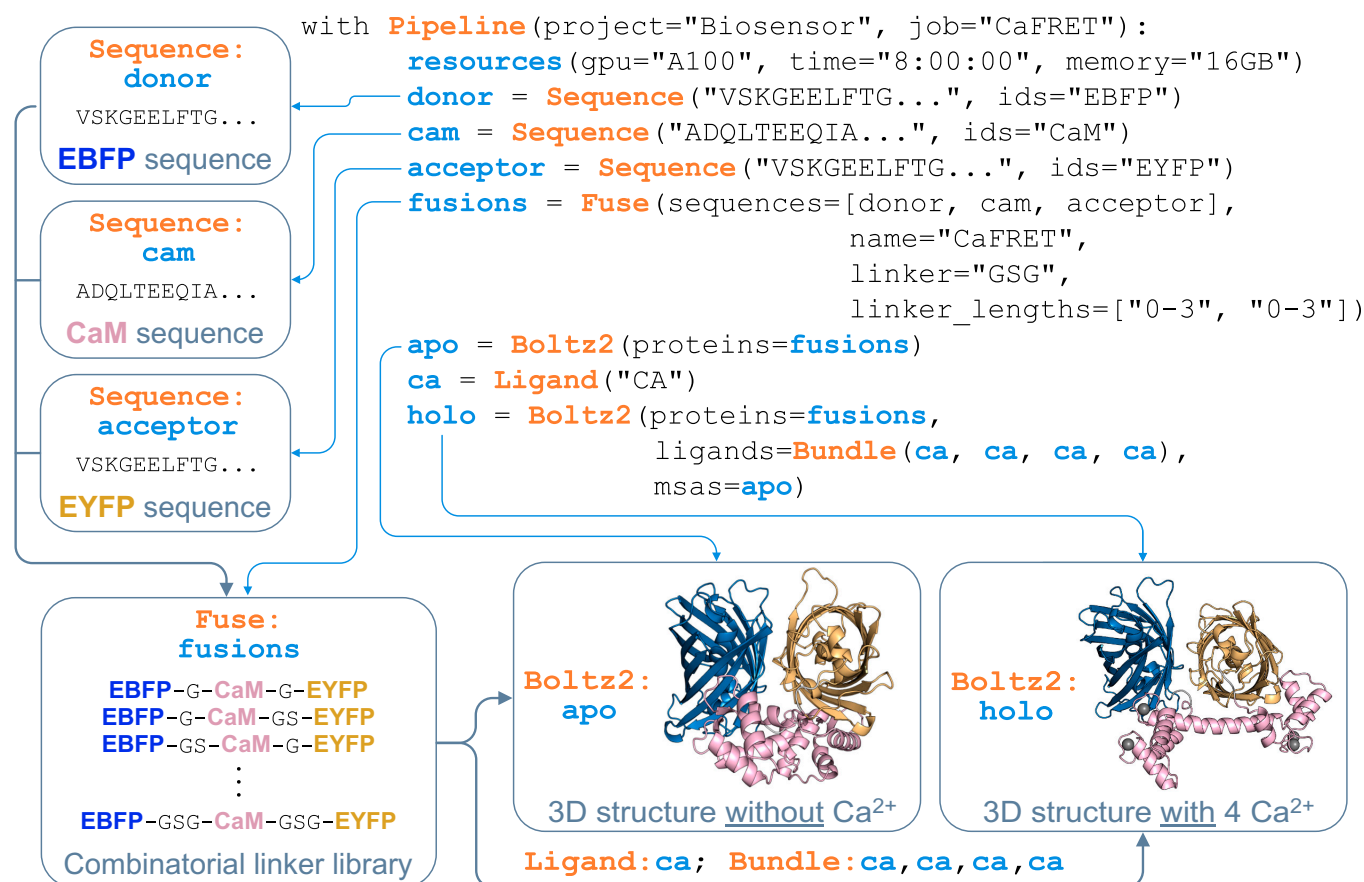


Fig. 5. Modeling a Förster resonance energy transfer (FRET)-based calcium sensor based on calmodulin and 2 fluorescent proteins. The server-intensive msas step is recycled for the holo structure determination from the apo calculations.

cycle then performs 5 steps: (a) the pocket is defined as all residues within 5 Å of the ligand in the current best structure, so the redesigned positions are recomputed at every cycle and can shift as the binding site evolves; (b) LigandMPNN generates 1,000 candidate sequences with only those pocket positions redesigned; (c) a mutation profile is built from the 1,000 sequences, recording per-position mutation frequencies relative to the current best; (d) 3 candidate mutants are then composed from this profile by weighted random sampling, with each mutant limited to at most 3 substitutions; and (e) the 3 candidates are cofolded with histpine, and the structure with the highest Boltz-2 binary binding probability across this set and the previous best is retained as the seed for the next cycle, for a total of 5 cycles. We adopt the mutation profile strategy to navigate only to low-distance mutants, and at each cycle we select the best candidate based on Boltz-2's binary classifier, which is consistent with it being more trustworthy than the affinity regression on this task.

This example illustrates the potential of BioPipelines to develop new computational methods for metric optimization using any desired choice of protein generation, inverse folding, protein prediction, small-molecule generation, docking method, or structural analysis tool.

Interactive prototyping

A key feature of BioPipelines is that the same workflow code can be tested prior to scaling up to inspect intermediate results, adjust parameters, and build the workflow step by step. Jupyter

notebooks allow a rich visual representation of the output and are an ideal platform for this purpose. All tools that produce structures (e.g., AlphaFold, Boltz-2, and RFdiffusion) automatically render an interactive py3Dmol 3D viewer. Images generated by the Plot tool are displayed inline together with the underlying data. These functionalities can be exploited in 2 ways, as illustrated in the Supplementary Materials, taking the second pipeline as an example (Fig. S7): by running the pipeline directly in Jupyter, one can inspect intermediate results step by step as soon as execution is completed, and by using the Load tool, one can inspect any tool output from a previously run pipeline. Importantly, the loaded output has the same structure as the original one, thus behaving in the same way, allowing the user to employ it as input in downstream tools.

Writing workflows and extending the framework

The motivation behind developing BioPipelines is that users with minimal coding experience can use and write custom workflows with ease. Because BioPipelines is a pure Python package with typed interfaces, users benefit from modern integrated development environment (IDE) features when writing workflows. Most modern IDEs (e.g., Visual Studio Code with Python and Pylance extensions) provide real-time autocompletion, parameter hints, and inline documentation for every tool and parameter. For example, typing "RFdiffusion(" immediately displays all available arguments with their types and descriptions, and passing an incorrect type triggers an underline warning before the script is ever run.

```

with Pipeline(project="NocT", job=f"IterativeOptimization"):
    resources(gpu="A100", time="24:00:00", memory="16GB")
    protein = PDB("5OT9")
    ligand = Ligand("Histopine")
    original = Boltz2(proteins=protein, ligands=ligand)
    current_best = Panda(tables=original.tables.affinity, pool=original)
    for cycle in range(5):
        suffix(f"Cycle{cycle+1}")
        pocket = DistanceSelector(structures=current_best,
                                ligand="LIG", distance=5)

        variants = LigandMPNN(structures=current_best, ligand="LIG",
                              num_sequences=1000,
                              redesigned=pocket.tables.selections.within)
        profile = MutationProfiler(original=current_best, mutants=variants)
        candidates = MutationComposer(frequencies=profile.tables.absolute_frequencies,
                                     num_sequences=10,
                                     mode="weighted_random", max_mutations=3)

        predicted = Boltz2(proteins=candidates, ligands=ligand)
        current_best = Panda(tables=[current_best.tables.result,
                                     predicted.tables.affinity],
                             operations=[Panda.concat(),
                                         Panda.sort("affinity_probability_binary"),
                                         Panda.head(1)],
                             pool=[current_best, predicted])

```

**PDB and ligand:
protein and ligand**
Input definitions

Iterative optimization cycles

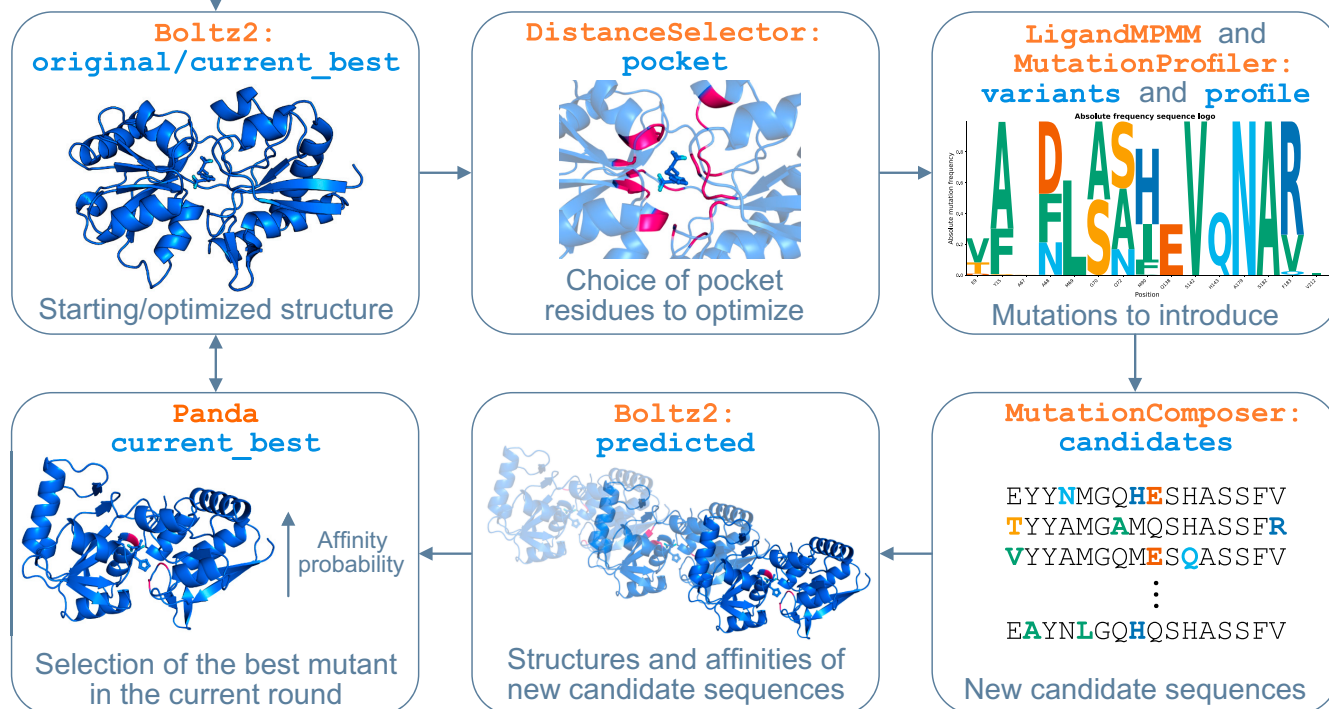


Fig. 6. Computational evolution of a binding site based on computed affinity.

The standardized tool interface also makes it straightforward to use AI coding assistants both to write pipelines against the existing toolset and to extend the framework with new tools.

To support this in a reproducible way, the repository ships a dedicated llm/ folder containing 4 working-agreement files that the user loads into a coding assistant (e.g., Claude Code and

Cursor) at the start of a session: pipelines.md for sessions that design and run a pipeline for a concrete biological problem, development.md for sessions that add or modify a tool wrapper, and cluster.md and colab.md for automated testing and debugging on a SLURM cluster or on Google Colab, respectively. Each prompt instructs the assistant to read the relevant documentation and conform to the framework standards.

Two worked examples are provided in the Supplementary Materials (Appendix A). Part 1 walks through the use of pipelines.md to design and run a pipeline starting from a stated biological objective, showing how a user without prior knowledge of the framework's internals can move from problem statement to a runnable script. Part 2 shows the addition of a new tool (ADMET-AI) [35] to the framework using development.md and automated debugging on a cluster and on Google Colab. Together, these examples make explicit what the prompt files take as given, where AI assistance is reliable, and where human review remains load-bearing. This approach dramatically reduces the effort required to keep the framework current with the rapidly evolving landscape of computational biology tools and enables individual laboratories to add specialized or in-house tools without deep familiarity with the framework internals.

Conclusions

BioPipelines makes computational protein and ligand design accessible to chemical biology laboratories by handling the computational logistics: environment management, file format conversion, cluster job scheduling via SLURM, and data tracking that would otherwise require dedicated bioinformatics support. Researchers define workflows as concise Python scripts that read as descriptions of the scientific experiment, prototype interactively in Jupyter notebooks, and submit the same code for production runs. The framework's support for compound library screening, iterative optimization, fusion-protein engineering, and end-to-end gene synthesis preparation addresses common needs in chemical biology. BioPipelines currently integrates over 40 tools, which we have organized by function in Table S1. New tools are added by implementing a standardized interface, detailed in the Supplementary Materials, and the framework provides base classes that handle environment activation, machine configuration, streams and tables access, completion tracking, and formatting. The codebase is written to be easily understood by AI coding agents, enabling nonexperts to program new pipelines or add more specialized tools with minimal effort.

BioPipelines is freely available under the MIT license at <https://github.com/locbp-uzh/biopipelines>. The repository includes all case-study workflows shown in this article, as Python scripts and pre-run Jupyter notebooks, with additional example pipelines. The full documentation can be found at <https://biopipelines.readthedocs.io>.

Acknowledgments

We thank all members of the Laboratory of Chemical and Biological Probes at University of Zurich for valuable feedback. Computational work was carried out in the ScienceCluster (S3IT) of the University of Zurich (Ubuntu 24.04, SLURM 25.05, NVIDIA L4/V100/A100/H100/H200) and on Google Colab (NVIDIA A100). Substantial parts of the code were written or refactored by Claude Code (Opus 4.6 and Opus 4.7).

Funding: This work was supported by the Swiss National Science Foundation through project grant 10004207.

Competing interests: The authors declare that they have no competing interests.

Supplementary Materials

Supplementary Text
Figs. S1 to S13
Tables S1 to S4
Appendices A and B

References

1. Simons KT, Bonneau R, Ruczinski I, Baker D. Ab initio protein structure prediction of CASP III targets using ROSETTA. *Proteins Struct Funct Genet*. 1999;37(S3):171–176.
2. Jumper J, Evans R, Pritzel A, Green T, Figurnov M, Ronneberger O, Tunyasuvunakool K, Bates R, Židek A, Potapenko A, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*. 2021;596(7873):583–589.
3. Abramson J, Adler J, Dunger J, Evans R, Green T, Pritzel A, Ronneberger O, Willmore L, Ballard AJ, Bambrick J, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*. 2024;630(8016):493–500.
4. Corley N, Mathis S, Krishna R, Bauer MS, Thompson TR, Ahern W, Kazman MW, Brent RI, Didi K, Kubaney A, etc. Accelerating biomolecular modeling with AtomWorks and RF3". *bioRxiv*. 2025. <https://doi.org/10.1101/2025.08.14.670328>
5. Chai Discovery team, Boitreaud J, Dent J, McPartlon M, Meier J, Reis V, Rogozhnikov A, Wu K. Chai-1: Decoding the molecular interactions of life. *BioRxiv*. 2024. <https://doi.org/10.1101/2024.10.10.615955>
6. Passaro S, Corso G, Wohlwend J, Reveiz M, Thaler S, Somnath VR, Getz N, Portnoi T, Roy J, Stark H, et al. Boltz-2: Towards accurate and efficient binding affinity prediction. *bioRxiv*. 2025. <https://doi.org/10.1101/2025.06.14.659707>
7. Watson JL, Juergens D, Bennett NR, Trippe BL, Yim J, Eisenach HE, Ahern W, Borst AJ, Ragotte RJ, Milles LF, et al. De novo design of protein structure and function with RFdiffusion. *Nature*. 2023;620(7976):1089–1100.
8. Ingraham JB, Baranov M, Costello Z, Barber KW, Wang W, Ismail A, Frappier V, Lord DM, Ng-Thow-Hing C, Van Vlack ER, et al. Illuminating protein space with a programmable generative model. *Nature*. 2023;623(7989):1070–1078.
9. Yim J, Trippe BL, Bortoli VD, Mathieu E, Doucet A, Barzilay R, Jaakkola T. SE(3) diffusion model with application to protein backbone generation. *arXiv*. 2023. <https://doi.org/10.48550/arXiv.2302.02277>
10. Dauparas J, Anishchenko I, Bennett N, Bai H, Ragotte RJ, Milles LF, Wicky BIM, Courbet A, de Haas RJ, Bethel N, et al. Robust deep learning-based protein sequence design using ProteinMPNN. *Science*. 2022;378(6615):49–56.
11. Dauparas J, Lee GR, Pecoraro R, An L, Anishchenko I, Glasscock C, Baker D. Atomic context-conditioned protein sequence design using LigandMPNN. *Nat Methods*. 2025;22(4):717–723.
12. Mirdita M, Schütze K, Moriwaki Y, Heo L, Ovchinnikov S, Steinegger M. ColabFold: Making protein folding accessible to all. *Nat Methods*. 2022;19(6):679–682.

13. Prihoda D, Ancona M, Calounova T, Kral A, Polak L, Hrbán H, Dickens NJ, Bitton DA. Ovo, an open-source ecosystem for *de novo* protein design. *bioRxiv*. 2025. <https://doi.org/10.1101/2025.11.27.691041>
14. Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. *Nat Biotechnol*. 2017;35(4):316–319.
15. Silke D, Iskander J, Pan J, Thompson AP, Papenfuss AT, Lucet IS, Hardy JM. ProteinDJ: A high-performance and modular protein design pipeline. *Protein Sci*. 2026;35(2):Article e70464.
16. Braun M, Tripp A, ProtFlow: A Python package to manage protein design workflows on computing clusters and local machines, version v0.1.0, GitHub (2024); <https://github.com/mabr3112/ProtFlow>
17. Yoo AB, Jette MA, Grondona M. SLURM: Simple Linux Utility for Resource Management. In: Feitelson D, Rudolph L, Schwiegelshohn U, editors. *Job scheduling strategies for parallel processing*. Berlin (Germany): Springer; 2003. p. 44–60.
18. Masters MR, Mahmoud AH, Lill MA. Investigating whether deep learning models for co-folding learn the physics of protein-ligand interactions. *Nat Commun*. 2025;16(1):8854.
19. Morehead A, Giri N, Liu J, Neupane P, Cheng J. Assessing the potential of deep learning for protein–ligand docking. *Nat Mach Intell*. 2026;8(1):32–41.
20. Buttenschoen M, Morris GM, Deane CM. PoseBusters: AI-based docking methods fail to generate physically valid poses or generalise to novel sequences. *Chem Sci*. 2024;15(9):3130–3139.
21. Chen LY, Lu WL, Pathania T, Chu IH, Ho MR, Chuang WC, Lou YC, Hung TI, Miyanoiri Y, Chang CE, et al. Mesostuctured water enhances stability of ProteinMPNN-designed ubiquitin-fold proteins. *J Am Chem Soc*. 2026;148(7):7363–7377.
22. Berman HM, Westbrook J, Feng Z, Gilliland G, Bhat TN, Weissig H, Shindyalov IN, Bourne PE. The Protein Data Bank. *Nucleic Acids Res*. 2000;28(1):235–242.
23. Alexaki A, Kames J, Holcomb DD, Athey J, Santana-Quintero LV, Lam PVN, Hamasaki-Katagiri N, Osipova E, Simonyan V, Bar H, et al. Codon and codon-pair usage tables (CoCoPUTs): Facilitating genetic variation analyses and recombinant gene design. *J Mol Biol*. 2019;431(13):2434–2441.
24. The Pandas Development Team, Pandas, version v3.0.3, Zenodo (2026); <https://doi.org/10.5281/ZENODO.3509134>
25. Bret G, Sindt F, Rognan D. Assessing Boltz-2 performance for the binding classification of docking hits. *J Chem Inf Model*. 2026;66(3):1511–1521.
26. Nittinger E, Yoluk Ö, Tibo A, Olanders G, Tyrchan C. Co-folding, the future of docking—Prediction of allosteric and orthosteric ligands. *Artif Intell Life Sci*. 2025;8:Article 100136.
27. Wu Z, Kan SBJ, Lewis RD, Wittmann BJ, Arnold FH. Machine learning-assisted directed protein evolution with combinatorial libraries. *Proc Natl Acad Sci*. 2019;116(18):8852–8858.
28. Biswas S, Khimulya G, Alley EC, Esvelt KM, Church GM. Low-*N* protein engineering with data-efficient deep learning. *Nat Methods*. 2021;18(4):389–396.
29. Khan A, Cowen-Rivers AI, Grosnit A, Deik DG, Robert PA, Greiff V, Smorodina E, Rawat P, Akbar R, Dreczkowski K, et al. Toward real-world automated antibody design with combinatorial Bayesian optimization. *Cell Rep Methods*. 2023;3(1):Article 100374.
30. Stanton S, Maddox WJ, Gruver N, Maffettone P, Delaney E, Greenside P, Wilson AG. Accelerating Bayesian optimization for biological sequence design with denoising autoencoders. *arXiv*. 2022. <https://doi.org/10.48550/arXiv.2203.12742>
31. Hie BL, Shanker VR, Xu D, Bruun TUJ, Weidenbacher PA, Tang S, Wu W, Pak JE, Kim PS. Efficient evolution of human antibodies from general protein language models. *Nat Biotechnol*. 2024;42(2):275–283.
32. Jiang K, Yan Z, Di Bernardo M, Sgrizzi SR, Villiger L, Kayabolen A, Kim BJ, Carscadden JK, Hiraizumi M, Nishimasu H, et al. Rapid in silico directed evolution by a protein language model with EVOLVEpro. *Science*. 2025;387(6732):eadr6006.
33. Yang J, Lal RG, Bowden JC, Astudillo R, Hameedi MA, Kaur S, Hill M, Yue Y, Arnold FH. Active learning-assisted directed evolution. *Nat Commun*. 2025;16(1):714.
34. Angermueller C, Dohan D, Belanger D, Deshpande R, Murphy K, Colwell L. Model-based reinforcement learning for biological sequence design. *ICLR 2020: The Eighth International Conference on Learning Representations*; 2020 Apr 26–May 1; virtual.
35. Swanson K, Walther P, Leitz J, Mukherjee S, Wu JC, Shivnaraine RV, Zou J. ADMET-AI: A machine learning ADMET platform for evaluation of large-scale chemical libraries. *Bioinformatics*. 2024;40(7):btac416.

BioPipelines: Accessible Computational Protein and Ligand Design for Chemical Biologists

Gianluca Quargnali and Pablo Rivera-Fuentes

Citation: Quargnali G, Rivera-Fuentes P. BioPipelines: Accessible Computational Protein and Ligand Design for Chemical Biologists. *Comput Struct Biotechnol J*. 2026;**35** (1):0129. DOI: 10.34133/csbj.0129

Deep learning methods for protein structure generation, sequence design, and structure and property prediction have created unprecedented opportunities for protein engineering and drug discovery. However, using these tools often requires navigating incompatible software environments, diverse input/output formats, and high-performance computing infrastructure, any of which may hinder adoption by primarily experimental chemical biology laboratories. Here, we present BioPipelines, an open-source Python framework that allows researchers to define multistep computational design workflows in a few lines of code. Additionally, its robust yet modular architecture provides a straightforward way to expand the tool kit with different functionalities, particularly by leveraging coding agents, with little effort. The framework currently integrates over 40 tools encompassing structure generation, sequence design, structure prediction, compound screening, and analysis. The same workflow code can be prototyped interactively in a Jupyter notebook and then submitted for production-scale runs without modification. We demonstrate applications in inverse folding, gene synthesis, *de novo* protein design, compound library screening, iterative binding site optimization, and fusion-protein linker optimization. We hope that this framework will empower researchers, allowing them to focus on the scientific question rather than computational logistics. BioPipelines is available under the MIT license at <https://github.com/locbp-uzh/biopipelines>.

Image

View the article online

<https://spj.science.org/doi/10.34133/csbj.0129>

Use of this article is subject to the [Terms of Service](#)

Computational and Structural Biotechnology Journal (ISSN 2001-0370) is published by the American Association for the Advancement of Science, 1200 New York Avenue NW, Washington, DC 20005.

Copyright © 2026 Gianluca Quargnali and Pablo Rivera-Fuentes.

No claim to original U.S. Government Works. Distributed under a [Creative Commons Attribution License \(CC BY 4.0\)](#).